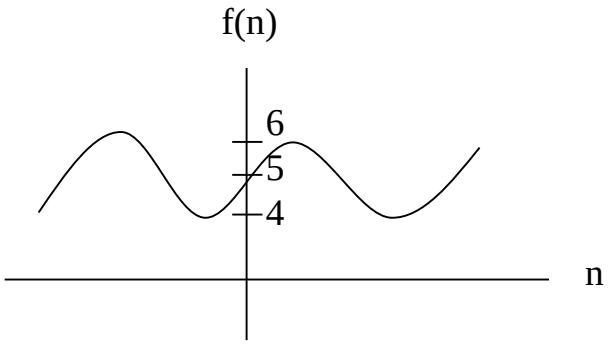
Growth Rates

- Example:
  - $f(n) = 5 + \sin n$



$f(n) \leq c * 1$, for $n \geq n_0$ $\quad (c = 6\ , n_0 = 1)$ $\quad \Rightarrow \quad f(n) \in O(1)$

$c * 1 \leq f(n)$, for $n \geq n_0$ $\quad (c = 1\ , n_0 = 1)$ $\quad \Rightarrow \quad f(n) \in \Omega(1)$

$\Rightarrow \quad f(n) \in \theta(1)$

Normally, as $n \nearrow$, $f(n) \nearrow$

Growth Rates

- Example:
  - Prove that $\sum_{k=1}^{n} k^p \in \theta(n^{p+1})$, for a fixed constant $p > 1$

    convert into closed-form

  - Upper bound:

$$\sum_{k=1}^{n} k^p = 1^p + 2^p + \cdots + n^p \leq n^p + n^p + \cdots + n^p = n * n^p = n^{p+1}$$

$$\Rightarrow \sum_{k=1}^{n} k^p \in O(n^{p+1})$$

Growth Rates

- Example:
  - Prove that $\sum_{k=1}^{n} k^p \in \theta(n^{p+1})$, for a fixed constant $p > 1$

    convert into closed-form

  - Lower bound:

$$\sum_{k=1}^{n} k^p = \sum_{k=1}^{\lfloor n/2 \rfloor} k^p + \sum_{k=\lfloor n/2 \rfloor+1}^{n} k^p \geq \sum_{k=\lfloor n/2 \rfloor+1}^{n} k^p$$

$$= (\lfloor n/2 \rfloor + 1)^p + (\lfloor n/2 \rfloor + 2)^p + \cdots + n^p \geq \left(\frac{n}{2}\right) * \left(\frac{n}{2}\right)^p = \frac{n^{p+1}}{2^{p+1}}$$

$\lceil n/2 \rceil$ terms : $\geq \left(\frac{n}{2}\right)$

Each term : $\geq \left(\frac{n}{2}\right)^p$

$$\Rightarrow \sum_{k=1}^{n} k^p \in \Omega(n^{p+1})$$

$$\boxed{\Rightarrow \sum_{k=1}^{n} k^p \in \theta(n^{p+1})}$$

Growth Rates

- Example:
  - $f(n) = \frac{n(n+1)}{2}$, $g(n) = n^2$

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{n^2+n}{2n^2} = \lim_{n\to\infty} \frac{1+1/n}{2} = \frac{1}{2}$$

$$\Rightarrow \mathrm{f}(n) \in \theta\big(g(n)\big)$$

Growth Rates

- Example:
  - $P(n)$ is a polynomial of degree k, where the coefficient of $n^k > 0$
    $\{P(n) = c_k n^k + c_{k-1} n^{k-1} + \cdots + c_1 n + c_o\}$

    Show that $P(n) \in \theta(n^k)$

Growth Rates

- Example:
  - Any polynomial function has smaller order than any exponential function
    $n^k \in o(a^n)$ --- $O(n^k) \subset O(a^n)$, for $k > 0, a > 1$

$$\lim_{n\to\infty} \frac{n^k}{a^n} = \lim_{n\to\infty} \frac{k*n^{k-1}}{(\ln a)*a^n} = \lim_{n\to\infty} \frac{k*(k-1)*n^{k-2}}{(\ln a)^2*a^n} = \cdots = \lim_{n\to\infty} \frac{k!}{(\ln a)^k*a^n} = 0$$

$$\Rightarrow n^k \in o(a^n)$$

Growth Rates

- Example:
  - $n!$ and 2^n

$$\lim_{n\to\infty} \frac{n!}{2^n} =$$

$$n! \cong \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \text{ , for large values of } n$$

$$= \lim_{n\to\infty} \frac{\sqrt{2\pi n}\left(\frac{n}{e}\right)^n}{2^n} = \lim_{n\to\infty} \sqrt{2\pi n}\left(\frac{n}{2e}\right)^n = \infty$$

$$\Rightarrow 2^n \in o(n!)$$

Integration Technique

- Complexity in terms of summation
  - $f(n) = \sum_{i=1}^{n} g(i)$ $\Longrightarrow$ Difficult to compute



$\int \quad \leq f(n) \leq \int$



closed-form $\leq f(n) \leq$ closed-form



$f(n) \in \theta(\quad)$

Integration Technique

- Complexity in terms of summation
  - $f(n) = \sum_{i=1}^{n} g(i)$

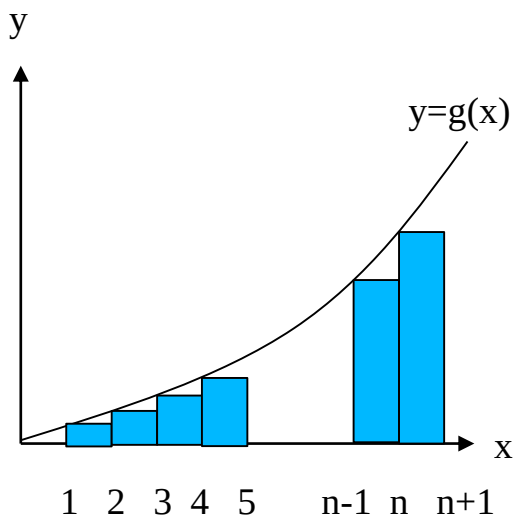
    easy summation ⇒ compute directly
  - Example:

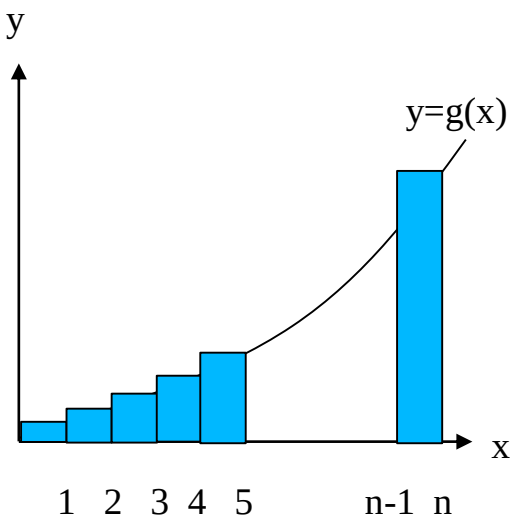
$$f(n) = \sum_{i=1}^{n} i$$

$$\Rightarrow f(n) = \frac{n(n+1)}{2} \in \theta(n^2)$$

Integration Technique

- Complexity in terms of summation
  - $f(n) = \sum_{i=1}^{n} g(i)$, where g is a nondecreasing function





Sum:

$g(1) * 1 + g(2) * 1 + \cdots + g(n) * 1$

$= \sum_{i=1}^{n} g(i) = f(n)$

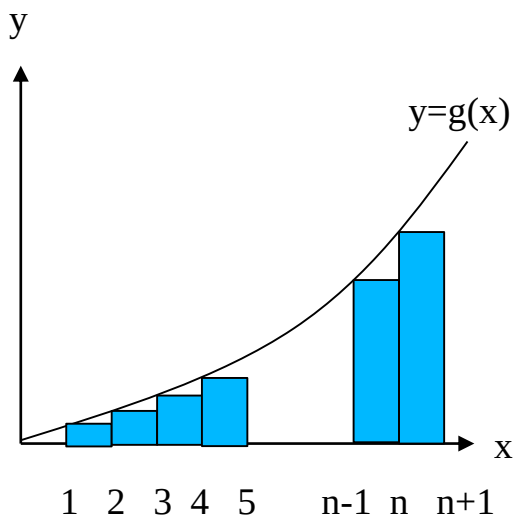
Sum:

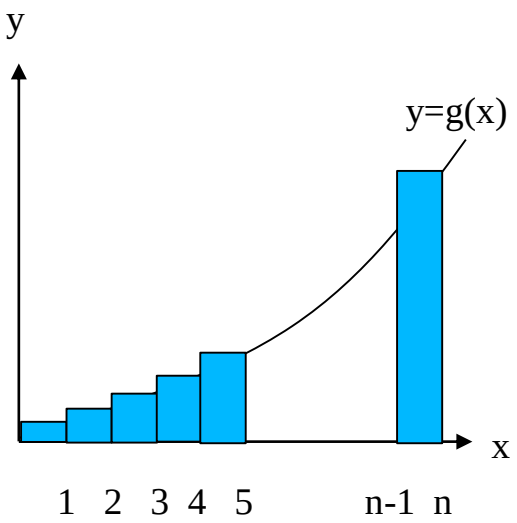
$g(1) * 1 + g(2) * 1 + \cdots + g(n) * 1$

$= \sum_{i=1}^{n} g(i) = f(n)$

Integration Technique

- Complexity in terms of summation
  - $f(n) = \sum_{i=1}^{n} g(i)$, where g is a nondecreasing function





Area:

$\int_{1}^{n+1} g(x)dx$

Greater than areas of rectangles

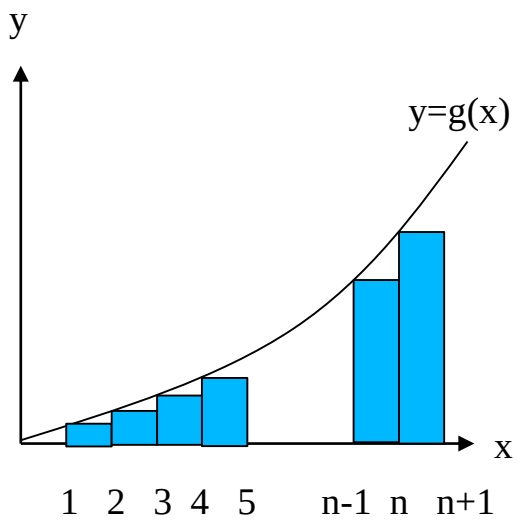
Area:

$\int_{0}^{n} g(x)dx$

Smaller than areas of rectangles

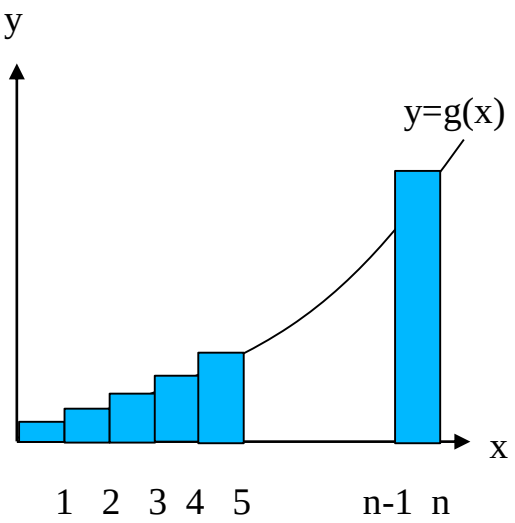
Integration Technique

- Complexity in terms of summation
  - $f(n) = \sum_{i=1}^{n} g(i)$, where g is a nondecreasing function



Thus:

$\sum_{i=1}^{n} g(i) \leq \int_{1}^{n+1} g(x)dx$



Thus:

$\int_{0}^{n} g(x)dx \leq \sum_{i=1}^{n} g(i)$

Integration Technique

- Complexity in terms of summation
  - $f(n) = \sum_{i=1}^{n} g(i)$, where g is a nondecreasing function

$$\int_0^n g(x)dx \leq f(n) \leq \int_1^{n+1} g(x)dx$$

  - Nonincreasing functions?
  - $\sum_{i=1}^{n} g(i)$

Integration Technique

- Example:
  - $f(n) = \sum_{i=1}^{n} i^2$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} \in \theta(n^3)$$

  - i^2 is increasing for $i \geq 1$

$$\int_0^n x^2 dx \leq f(n) \leq \int_1^{n+1} x^2 dx$$

$$\Rightarrow \frac{x^3}{3}\Big|_0^n \leq f(n) \leq \frac{x^3}{3}\Big|_1^{n+1}$$

$$\Rightarrow \frac{n^3}{3} \leq f(n) \leq \frac{(n+1)^3-1}{3}$$

$$\Rightarrow f(n) \sim \frac{n^3}{3} \quad \Longrightarrow \quad f(n) \in \theta(n^3)$$

Integration Technique

- Example: Sum of logarithms
  - $L(n) = \sum_{i=1}^{n} \log i \quad (= \log n!)$

O and Ω $\Rightarrow$ θ

  - $L(n) = \sum_{i=1}^{n} \log i = \log 1 + \log 2 + \cdots + \log n \leq n \log n$

    $\Rightarrow L(n) \in O(n \log n)$

    $L(n) = \sum_{i=1}^{n} \log i = \sum_{k=1}^{\lfloor n/2 \rfloor} \log i + \sum_{k=\lfloor n/2 \rfloor+1}^{n} \log i \geq \sum_{k=\lfloor n/2 \rfloor+1}^{n} \log i$

    $= \log\left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right) + \log\left(\left\lfloor \frac{n}{2} \right\rfloor + 2\right) + \cdots + \log n$

    $\geq \left(\frac{n}{2}\right) \log\left(\frac{n}{2}\right) = \left(\frac{n}{2}\right)(\log n - \log 2) \geq \left(\frac{n}{2}\right)\left(\log n - \frac{\log n}{2}\right) = \left(\frac{n}{2}\right)\left(\frac{\log n}{2}\right) = \frac{n \log n}{4}$

    $\Rightarrow L(n) \in \Omega(n \log n)$

$\Longrightarrow \quad L(n) \in \theta(n \log n)$

Integration Technique

- Example: Sum of logarithms
  - $L(n) = \sum_{i=1}^{n} \log i \quad (= \log n!)$

Integration

  - $\int_0^n \log x \, dx \le L(n) \le \int_1^{n+1} \log x \, dx$

$$\int \ln x \, dx = x \ln x - x$$

$$\Rightarrow x \ln x - x|_0^n \le L(n) \le x \ln x - x|_1^{n+1}$$

$$\lim_{x \to 0} x \ln x = \lim_{x \to 0} \frac{\ln x}{1/x} = \lim_{x \to 0} \frac{1/x}{-1/x^2} = \lim_{x \to 0} -x = 0$$

$$\Rightarrow n \ln n - n \le L(n) \le (n+1) \ln(n+1) - n$$

$$\Rightarrow L(n) \in \theta(n \log n)$$

$$\lim_{n \to \infty} \frac{n \ln n - n}{(n+1) \ln(n+1) - n} = 1 \qquad [(\ln f(x))' = \frac{f'(x)}{f(x)}]$$

$$\Rightarrow L(n) \in \sim(n \log_a n)$$

Integration Technique

- Example: Sum of logarithms
  - $L(n) = \sum_{i=1}^{n} \log i \quad (= \log n!)$

Integration

$$n \ln n - n \leq L(n) \leq (n+1)\ln(n+1) - n$$

$$\Rightarrow e^{n \ln n - n} \leq e^{\log n!} \leq e^{(n+1)\ln(n+1) - n}$$

$$\Rightarrow \frac{e^{n \ln n}}{e^n} \leq n! \leq \frac{e^{(n+1)\ln(n+1)}}{e^n}$$

$a^{n \log_a n} = n^n$

$$\Rightarrow \frac{n^n}{e^n} \leq n! \leq \frac{(n+1)^{(n+1)}}{e^n}$$

Bound for $n!$ in terms of $\left(\frac{n}{e}\right)^n$

~ Stirling's formula

Integration Technique

- Example: Harmonic series
  - $H(n) = \sum_{i=1}^{n} \frac{1}{i} \quad (= 1 + \frac{1}{2} + \cdots + \frac{1}{n})$

Can we use the formula?

$$\int_0^n g(x)dx \le f(n) \le \int_1^{n+1} g(x)dx$$

  - $f(n) = \sum_{i=1}^{n} g(i)$, where g is a nonincreasing function

$$\int_1^{n+1} g(x)dx \le f(n) \le \int_0^n g(x)dx$$

Integration Technique

- Example: Harmonic series
  - $H(n) = \sum_{i=1}^{n} \frac{1}{i} \quad (= 1 + \frac{1}{2} + \cdots + \frac{1}{n})$
  - $\int_1^{n+1} g(x)dx \le H(n) \le \int_0^n g(x)dx$

    $\Rightarrow \int_1^{n+1} \frac{1}{x} dx \le H(n) \le \int_0^n \frac{1}{x} dx$

    $\Rightarrow \ln x|_1^{n+1} \le H(n) \le \ln x|_0^n$

    $\Rightarrow \ln(n+1) \le H(n) \le \infty$

Integration Technique

- Example: Harmonic series
  - $H(n) = \sum_{i=1}^{n} \frac{1}{i} \quad (= 1 + \frac{1}{2} + \cdots + \frac{1}{n})$

$$\Rightarrow \ln(n+1) \le H(n) \le \infty$$

$$H(n) = 1 + \sum_{i=2}^{n} \frac{1}{i} \le 1 + \int_1^n \frac{1}{x} dx = 1 + \ln x \Big|_1^n = 1 + \ln n$$

$$\Rightarrow \ln(n+1) \le H(n) \le \ln n + 1$$

$$\Rightarrow \ln n \le H(n) \le 2 \ln n$$

$$\Rightarrow H(n) \in \theta(\log n)$$

$$\lim_{n\to\infty} \frac{\ln(n+1)}{\ln n+1} = \lim_{n\to\infty} \frac{1/n+1}{1/n} = \lim_{n\to\infty} \frac{n}{n+1} = 1$$

$$\Rightarrow H(n) \in \sim(\log_e n)$$

Linear Search

```
function LinearSearch (L[0:n-1], X)
   for i←0 to n-1 do
      if X=L[i] then
         return (i)
      endif
   endfor
   return (-1)
end
```

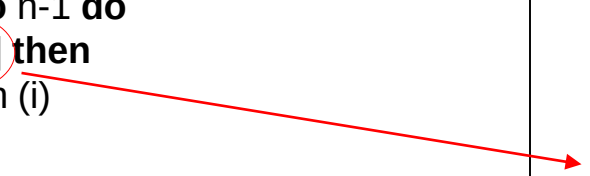
Follow
our pseudocode convention
for sequential & parallel algorithms
Textbook, Appendix G

- Comparison-based searching algorithm

  Making comparisons involving list elements and making decisions based on these comparisons

Linear Search

```
function LinearSearch (L[0:n-1], X)
   for i←0 to n-1 do
      if X=L[i] then
         return (i)
      endif
   endfor
   return (-1)
end
```



basic operation

- Basic operation

  Comparison

- Not basic operation

  Loop incrementation – not typical/characteristic

  Return statement – not repetitive

  …

Linear Search

```
function LinearSearch (L[0:n-1], X)
   for i←0 to n-1 do
      if X=L[i] then
         return (i)
      endif
   endfor
   return (-1)
end
```

- $B(n)$
  - $X = L[0]$
  - $\Rightarrow B(n) = 1 \in \theta(1)$ $\Longrightarrow$ constant complexity

Linear Search

```
function LinearSearch (L[0:n-1], X)
   for i←0 to n-1 do
      if X=L[i] then
         return (i)
      endif
   endfor
   return (-1)
end
```

- $W(n)$
  - $X = L[n-1]$ or $X \notin L$
  - $\Rightarrow W(n) = n \in \theta(n)$ $\Longrightarrow$ linear complexity

Linear Search

```
function LinearSearch (L[0:n-1], X)
  for i←0 to n-1 do
    if X=L[i] then
      return (i)
    endif
  endfor
  return (-1)
end
```

- $A(n)$
  - First thing we need?

Probability distribution

  - Assume that probability of successful search : p $(0 \leq p \leq 1)$
  - Assume that it is equally likely that $X = L[i]$ for all i

And for simplicity

  - Assume elements in L are distinct

Linear Search

```
function LinearSearch (L[0:n-1], X)
   for i←0 to n-1 do
      if X=L[i] then
         return (i)
      endif
   endfor
   return (-1)
end
```

- $A(n)$
  - $A(n) = \sum_{i=B(n)}^{W(n)} i * p_i = \sum_{i=1}^{n} i * p_i$

probability that *i* operations will be executed

$$p_i = \begin{cases} \frac{p}{n} & , for\ 1 \le i \le n-1 \\ \frac{p}{n} + (1-p) & , for\ i = n \end{cases}$$

$$A(n) = \sum_{i=1}^{n-1} i * \frac{p}{n} + n\left(\frac{p}{n} + (1-p)\right) = \left(1 - \frac{p}{2}\right)n + \frac{p}{2}$$

$\Rightarrow A(n) \in \theta(n) \Longrightarrow$ Linear complexity

Linear Search

```
function LinearSearch (L[0:n-1], X)
   for i←0 to n-1 do
      if X=L[i] then
         return (i)
      endif
   endfor
   return (-1)
end
```

- $A(n)$
  - $A(n) = \left(1 - \frac{p}{2}\right)n + \frac{p}{2}$

Reasonable?

- $p = 1 \Rightarrow A(n) = \frac{n+1}{2}$
- $p = 0 \Rightarrow A(n) = n$

Binary Search

- $B(n)$
  - $X = L[\frac{n}{2}]$
  - $\Rightarrow B(n) = 1 \in \theta(1)$ constant complexity

Binary Search

- $W(n)$
  - Not easy to find for general n
  - Assume $n = 2^k - 1 \quad (k \geq 1)$
  - $X \notin L$ or (e.g.) $X = L[0]$

Compare X with $L[2^{k-1} - 1]$, $L[2^{k-2} - 1]$, …, $L[0]$

k comparisons

$n = 2^k - 1 \Rightarrow 2^k = n + 1 \Rightarrow k = \log_2(n + 1)$

$\Rightarrow W(n) = \log_2(n + 1) \in \theta(\log n)$, for $n = 2^k - 1$

Binary Search

- $W(n)$
  - $W(n) = \log_2(n+1)$, for $n = 2^k - 1$

    ⟹ $W(n) = \cdots$, for $n \in \mathbb{N}$
  - For any integer n, $2^{i-1} - 1 \le n \le 2^i - 1$

    $\Rightarrow 2^{i-1} \le n+1 \le 2^i \Rightarrow \log_2 2^{i-1} \le \log_2(n+1) \le \log_2 2^i \Rightarrow i - 1 \le \log_2(n+1) \le i$
  - $W(n)$ is increasing $\Rightarrow W\left(2^{i-1} - 1\right) \le W(n) \le W\left(2^i - 1\right)$

    $\Rightarrow \log_2\left(2^{i-1} - 1 + 1\right) \le W(n) \le \log_2\left(2^i - 1 + 1\right)$

    $\Rightarrow i - 1 \le W(n) \le i$

$\log_2(n+1) - 1 \le W(n) \le \log_2(n+1) + 1$

$\Rightarrow W(n) \in \theta(\log n)$, for $n \in \mathbb{N}$

Interpolation

Binary Search

- $A(n)$

  - Assume that probability of successful search : p $(0 \leq p \leq 1)$

  For successful search:

  - Assume that it is equally likely that $X = L[i]$ for all i

  For unsuccessful search:

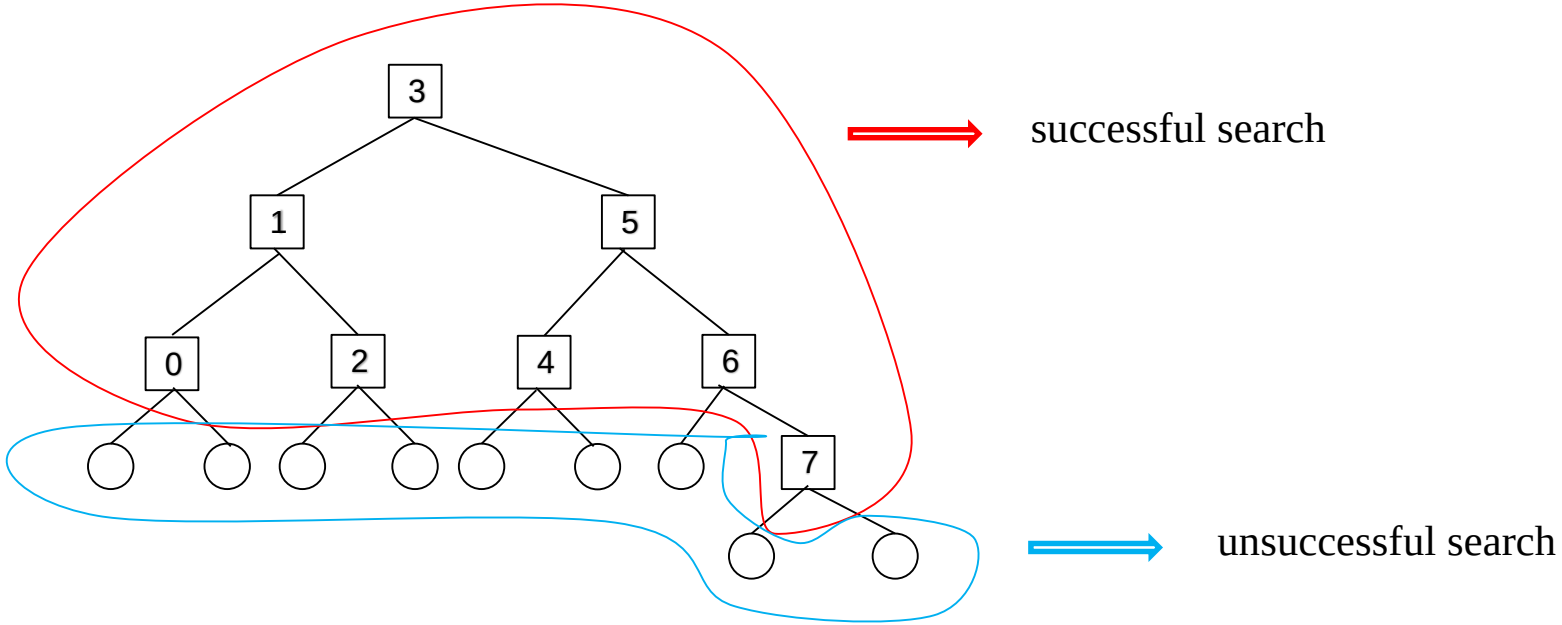
  - Assume that it is equally likely that X falls in any of the $n + 1$ intervals

    $X < L[0]$
    $L[0] < X < L[1]$
    $L[1] < X < L[2]$
    …
    $L[n-2] < X < L[n-1]$
    $L[n-1] < X$

Binary Search

- $A(n)$
  - Example: $L[0:7]$

    Execution of the algorithm – an extended tree



all possible executions of the algorithm

Binary Search

- $A(n)$
  - Example: $L[0:7]$

    Execution of the algorithm – an extended tree

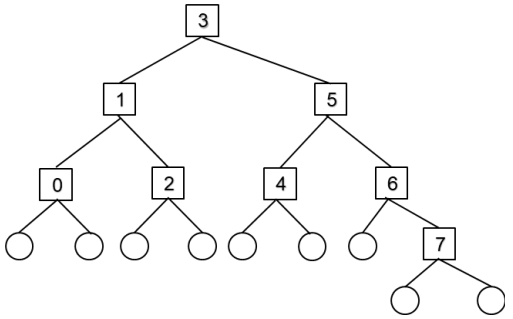
  - $A(n) = \sum_{I \in \mathrm{T}_n} \tau(I) * p(I)$

Binary Search

- $A(n)$

  - Example: $L[0:7]$

    Execution of the algorithm – an extended tree



  - Successful case:

    Probability that $X = L[i]\ (0 \le i \le n-1) : \frac{p}{n}$

$$A(n) = \sum_{I \in \mathrm{T}_n} \tau(I) * p(I)$$
$$= \frac{p}{n} * 1 + \frac{p}{n} * 2 + \frac{p}{n} * 2 + \frac{p}{n} * 3 + \frac{p}{n} * 3 + \cdots$$
$$= \frac{p}{n} * (1 + 2 + 2 + 3 + 3 + \cdots) = \frac{p}{n} * (n + 0 + 1 + 1 + 2 + 2 + \cdots)$$

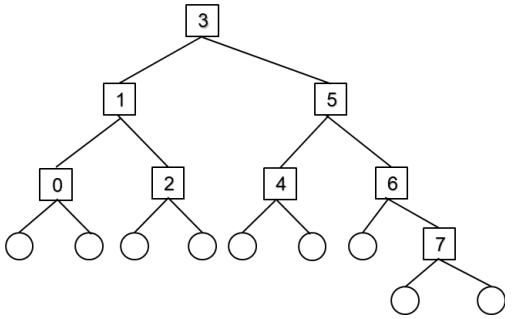
internal path length (IPL):
sum of the lengths of the paths from the root to the internal nodes

Binary Search

- $A(n)$

  - Example: $L[0:7]$

    Execution of the algorithm – an extended tree



  - Unsuccessful case:

    Probability that X falls in any of the $n+1$ intervals : $\frac{1-p}{n+1}$

$$A(n) = \sum_{I \in \mathrm{T}_n} \tau(I) * p(I)$$
$$= \frac{1-p}{n+1} * 3 + \frac{1-p}{n+1} * 3 + \frac{1-p}{n+1} * 3 + \cdots$$
$$= \frac{1-p}{n+1} * (3 + 3 + 3 + \cdots)$$

leaf path length (LPL):
sum of the lengths of the paths from the root to the leaf nodes

Binary Search

- $A(n)$
  - Example: $L[0:7]$

    Execution of the algorithm – an extended tree

  - $A(n) = \sum_{I \in \mathrm{T}_n} \tau(I) * p(I)$

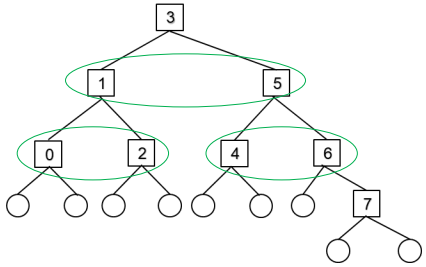
$$= \frac{p}{n} * \Big((IPL(T) + n)\Big) + \frac{1-p}{n+1} * \Big(\big(LPL(T)\big)\Big)$$

Binary Search

- Definitions

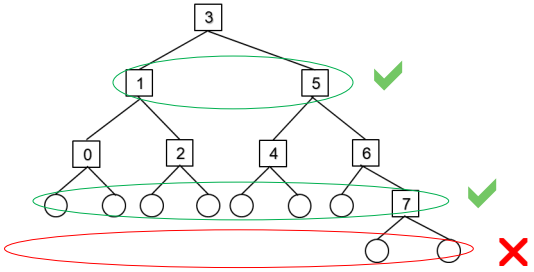
  - 2-tree:

    A binary tree such that every node that is not a leaf has exactly two children



  - Full binary tree at level i

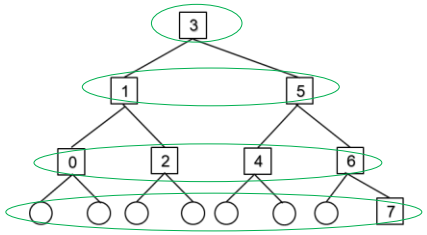
    A binary tree that contains the maximum number (2^i) of nodes at level i ($0 \leq i \leq D(T)$)



Binary Search

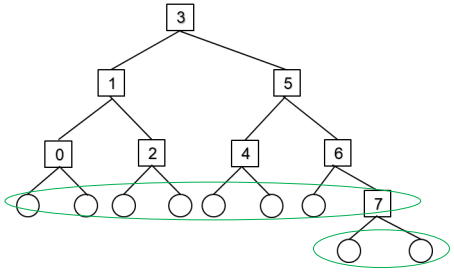
- Definitions
  - Full binary tree

    A binary tree that is full at every level i $(0 \leq i \leq D(T))$



  - Full at the second-deepest level

    A 2-tree in which all leaf nodes are in the levels $D(T) - 1$ and $D(T)$



Binary Search

- Lemmas
  - For a 2-tree T,

    $IPL(T) = LPL(T) - 2 * I$,

    where I is the number of internal nodes

  - For any binary tree T,

    $LPL(T) \geq L\lfloor \log_2 L \rfloor + 2\left(L - 2^{\lfloor \log_2 L \rfloor}\right)$,

    where L is the number of leaf nodes

    $=$ if 2-tree that is full at the second-deepest level

Binary Search

- $A(n)$

  - $A(n) = \sum_{I \in \mathrm{T}_n} \tau(I) * p(I)$

$$= \frac{p}{n} * \left((IPL(T) + n)\right) + \frac{1-p}{n+1} * \left(\left(LPL(T)\right)\right)$$

$$= \frac{p}{n} * \left((LPL(T) - n)\right) + \frac{1-p}{n+1} * \left(\left(LPL(T)\right)\right)$$

2-tree & n internal nodes

$$= \left(\frac{p}{n} + \frac{1-p}{n+1}\right) LPL(T) - p$$

$$= \left(\frac{p}{n} + \frac{1-p}{n+1}\right) \left[(n+1)\lfloor \log_2(n+1) \rfloor + 2\left((n+1) - 2^{\lfloor \log_2(n+1) \rfloor}\right)\right] - p$$

2-tree & full at the second-deepest level

$$\geq \left(\frac{p}{n} + \frac{1-p}{n+1}\right)(n+1)\lfloor \log_2(n+1) \rfloor - p$$

$\left((n+1) - 2^{\lfloor \log_2(n+1) \rfloor}\right) \geq 0$

$\Rightarrow A(n) \in \Omega(\log n)$

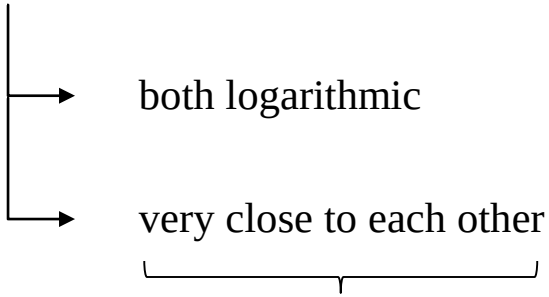
$A(n) \in O(\log n)$

For any algorithm, $A(n) \leq W(n)$ & $W(n) \in O(\log n)$

$\Rightarrow A(n) \in \theta(\log n)$

Binary Search

- $A(n)$ & $W(n)$



$$\log_2(n+1) - 1 \leq W(n) \leq \log_2(n+1) + 1$$

$$A(n) \geq \frac{n+1}{n} \lfloor \log_2(n+1) \rfloor - 1$$

for large *n*, differ with only 1-2 basic operations